

Relational vs Non-Relational Databases

Relational databases (SQL) store data in structured tables with rows and columns, where each table has a fixed schema and relationships between tables are enforced through keys (primary/foreign keys). They use SQL for queries and typically follow ACID properties (Atomicity, Consistency, Isolation, Durability), making them strong for consistency and complex joins.

Non-relational databases (NoSQL) store data in more flexible formats — documents, key-value pairs, wide-column stores, or graphs — without a fixed schema. They are built for scalability and flexibility, often trading strict consistency for speed and horizontal scaling (they usually follow BASE: Basically Available, Soft state, Eventual consistency instead of ACID).

Key Differences

Aspect	Relational (SQL)	Non-Relational (NoSQL)
Data structure	Tables (rows/columns)	Documents, key-value, columns, graphs
Schema	Fixed, predefined	Dynamic / flexible
Relationships	Enforced via foreign keys, joins	Usually embedded or handled in app logic
Scaling	Vertical (mostly)	Horizontal (built for it)
Consistency model	ACID (strong consistency)	BASE (eventual consistency, usually)
Query language	SQL	Varies (MongoDB query language, CQL, etc.)
Best for	Structured data, complex transactions (banking, ERP)	Unstructured/semi-structured data, high scale (real-time apps, big data)

Examples

Relational:

- PostgreSQL
- MySQL
- Microsoft SQL Server
- Oracle Database
- SQLite

- Snowflake (cloud data warehouse, SQL-based)

Non-relational:

- MongoDB (document store)
- Redis (key-value store)
- Cassandra (wide-column store)
- DynamoDB (key-value/document, AWS)
- Neo4j (graph database)
- Couchbase (document store)

Context Note

In the earlier Intelligent Data Processor Core implementation plan, PostgreSQL and Snowflake (both relational) were the storage targets mentioned, so that pipeline is designed around structured, schema-enforced data rather than NoSQL stores.